API testing with Postman and Newman: interview questions, answered with this project

Questions about API testing, Postman and Newman that come up in SDET and QA lead interviews. Where the project shows the idea, the answer ends with an **In the project** pointer. Links go to github.com/niranjankagri/postman-newman-api-testing.

Versions: Newman 6.2, Postman Collection format v2.1, Node.js 22.

Contents

1. API testing basics (Q1-Q15)
2. Postman building blocks (Q16-Q35)
3. Newman and CI (Q36-Q48)
4. Strategy and scenarios (Q49-Q60)

1. API testing basics

Q1. What do you check when you test an API?

The status code, the body, the headers, the behaviour on bad input, access rules, and the effect on stored data. A `200` alone proves little.

In the project: `TC-233` reads an order back and compares it with what the create call returned.

Q2. What is the difference between 401 and 403?

`401` means the server doesn't know who you are, so authenticate. `403` means it knows, and you aren't allowed. A client handles them differently: log in again, or stop.

In the project: `TC-221` and `TC-222`.

Q3. When is 400 right, and when 422?

`400`: the request couldn't be read, such as broken JSON. `422`: it was read, but the content breaks a rule. Many APIs use `400` for both. What matters is that the team picks one and tests it.

In the project: `TC-236` for broken JSON, `TC-241` for the rules.

Q4. Which HTTP methods are idempotent?

`GET`, `PUT`, `DELETE`, `HEAD` and `OPTIONS`: repeating them leaves the same state. `POST` usually isn't, which is why payment APIs add an idempotency key.

Q5. PUT or PATCH?

`PUT` replaces the whole resource, and `PATCH` changes part of it. A test for `PUT` should check that a field left out is really gone, or that the API refuses the request.

Q6. What does a good negative test look like?

One rule broken at a time, with the exact status and message asserted. If two things are wrong, the test can't show which rule fired.

In the project: each row of `invalid-orders.csv` breaks one rule and names the expected error.

Q7. What are boundary tests for an API?

The values on each side of a limit. For a quantity of 1 to 100: 0, 1, 100 and 101.

In the project: quantity 0 and 101 are refused (`TC-131`), and 100 is accepted with an exact total (`TC-132`).

Q8. What is contract testing compared with schema validation?

A schema checks the shape of one provider's answer. Consumer contract testing, as in Pact, checks that the provider gives each real consumer what it uses. A schema can pass while a consumer breaks.

In the project: `openapi.json` checks every response in a Newman run. The design document points to my Pact project for the other kind.

Q9. Why does the order of checks in an API matter?

The order decides what an attacker can learn. The token should be checked before the body, or an unauthenticated caller can probe the validation rules.

In the project: `TC-114` sends broken JSON with no token and expects `401`.

Q10. What should you test about authentication?

No token, a wrong token, an expired token, a token for another user, and each protected route, not just one. Reading is often forgotten when writing is protected.

In the project: `TC-113` and `TC-223` check that reading an order needs a token too.

Q11. How do you test an API that isn't finished?

Against a mock or a stub built from the agreed contract, so the tests are ready when the API is. Then run them against the real thing. My WireMock project shows this from the client's side.

Q12. What is the difference between functional and non-functional API tests?

Functional tests ask whether the answer is right. Non-functional tests ask about speed, load, security and reliability. A single response-time check is a sanity check, not a performance test.

In the project: `TC-200` allows a second per answer, and the design document says what that doesn't prove.

Q13. What headers do you check?

`Content-Type`, caching headers on personal data, security headers, and anything the contract promises, such as `Location` after a create or `Retry-After` on a `429`.

In the project: `TC-200` checks `Content-Type` and `Cache-Control: no-store` on every answer.

Q14. How would you test pagination?

First page, last page, one past the end, page size limits, a stable order, and whether an item added mid-way is skipped or repeated.

Q15. What makes API tests flaky?

Shared data, order dependence, timing assumptions, and environments that other people change. The fix is usually isolation.

In the project: every run starts its own API on a free port, with an empty store.

2. Postman building blocks

Q16. What is a collection?

A set of requests in folders, with scripts, variables and auth settings. It's a JSON file, so it can live in Git and be reviewed.

In the project: `postman/collections/commerce-api.postman_collection.json`.

Q17. What are the variable scopes, from narrowest to widest?

Local, data, environment, collection, global. The narrowest one wins.

Q18. Where do you put each kind of value?

Environment: what changes per system, such as the address and tokens. Collection: what belongs to the flow. Data: what changes per iteration. Globals are rarely worth it.

In the project: `baseUrl` and `authToken` in the environment, `lastOrderId` in the collection, `quantity` in the CSV.

Q19. What happens if a variable isn't defined?

Postman sends the literal text `{{name}}`, without an error. A test can then pass for the wrong reason.

In the project: `TC-155` fails if any `{{variable}}` has no definition.

Q20. What is a pre-request script?

JavaScript that runs before the request, to prepare data or sign it. Test scripts run after the answer.

Q21. In what order do scripts run?

Pre-request scripts: collection, folder, request. Then the request. Then test scripts: collection, folder, request.

Q22. Why put a test on the collection?

It runs after every request, so a new request can't miss it.

In the project: `TC-200`.

Q23. How do you chain requests?

Save a value from one answer with `pm.collectionVariables.set`, then use `{{name}}` in the next request.

In the project: `TC-231` saves the order ID, and `GET read created order` uses it.

Q24. How do you validate a response against a JSON Schema?

`pm.response.to.have.jsonSchema(schema)`. Set `additionalProperties: false` if a new field should fail the test.

In the project: `TC-232`, with the schema taken from `openapi.json` at run time, so it isn't copied. That setting caught a deliberately leaked field that the unit tests missed.

Q25. What is the difference between `pm.expect` and `pm.test`?

`pm.test` names a check and records it. `pm.expect` is the assertion inside it. An `expect` outside a `test` isn't reported as a check.

Q26. Why does one `pm.test` stop at its first failed `expect`?

The failed assertion throws, so the rest of that test doesn't run. Separate concerns go in separate tests.

In the project: the contract check is its own `pm.test`, so a wrong status doesn't hide it.

Q27. How do you set auth once for a whole collection?

Set it on the collection, with requests inheriting it. A request can override it, for example with `noauth`.

In the project: bearer `{{authToken}}` on the collection, and `noauth` on `TC-221`.

Q28. How do you keep secrets out of a shared collection?

Use variables, keep real values out of exported files, and mark them as secret. Remember an exported environment contains its values.

In the project: `TC-154` fails if the token appears in the collection.

Q29. What is `pm.sendRequest` for?

Calling another request from a script, for example to fetch a token before the main request.

Q30. How do you control the order of requests?

They run from top to bottom. `pm.execution.setNextRequest` can jump or stop, but it makes a run harder to follow, so I use it rarely.

Q31. How do you share code between scripts?

Collection-level scripts, or a package library where the tool supports it. Copying the same script into many requests is the usual mistake.

Q32. What is a data file in the Collection Runner?

A CSV or JSON file with one iteration per row. Columns become variables, and scripts read them with `pm.iterationData`.

In the project: `customers.csv` and `invalid-orders.csv`.

Q33. How do you get the test name from the data?

Build it from the row, with a template string.

In the project: `TC-241 refused: ${rule}`, so a report shows which rule failed.

Q34. How do you avoid copied expected values?

Take them from an earlier answer in the run. A price copied into a test breaks when the catalog changes, for no good reason.

In the project: `TC-231` multiplies the price that `TC-211` saved.

Q35. What are Postman mock servers and monitors for?

Mock servers answer with saved examples, so a client can start early. Monitors run a collection on a schedule from Postman's cloud. This project runs in GitHub Actions instead, so it needs no account.

3. Newman and CI

Q36. What is Newman?

The command-line runner for Postman collections. It's a Node.js package, so a collection can run in CI without the app.

Q37. What is the basic command?

`newman run collection.json -e environment.json -d data.csv -r cli,junit`.

Q38. Why use Newman as a library rather than the command line?

To add logic around the run: start a server, run twice, and decide the result.

In the project: `scripts/run-newman.js` calls `newman.run()`.

Q39. How does Newman fail a build?

The command exits with 1 when a check fails. As a library, you read `summary.run.failures` and set the exit code yourself.

In the project: `run-newman.js` sets `process.exitCode`.

Q40. Can a Newman run pass without testing anything?

Yes. It counts only the checks that ran, so a skipped folder or an empty data file gives fewer checks and no failures.

In the project: each run has an expected assertion count. Emptying the rules CSV gives `2 assertions ran, expected 16`.

Q41. How do you run one folder?

`--folder "Order rules"`, which can be repeated.

Q42. Why does this project run the collection twice?

A run has one data file and repeats everything for each row. The rules and the main flow need different data.

Q43. How do you override a variable from the command line?

`--env-var name=value` for an environment variable, or `--global-var` for a global one.

In the project: `run-newman.js` passes the free port's address as `baseUrl`.

Q44. Which reports can Newman write?

`cli`, `json` and `junit` are built in. `html` and the widely used `htmlextra` are separate packages.

In the project: JUnit XML per run, kept as a CI artifact, and a Markdown table in the run summary.

Q45. How do you stop a slow request from hanging CI?

`--timeout-request`, plus a time limit on the CI job itself.

In the project: 5 seconds per request, and 10 minutes for the job.

Q46. What does `--bail` do?

It stops the run at the first failure. That's quicker, but you see one problem instead of all of them. I leave it off.

Q47. How do you keep a collection in Git without pain?

Export it in a stable format, review the diffs, and keep one owner per change. Scripts are stored as line arrays in JSON, so changes are readable, but merge conflicts are awkward.

Q48. How do you handle vulnerabilities in a tool's own dependencies?

Check whether the code reaches production, fix what you can with overrides, and write down the rest. Don't run `npm audit fix --force` without reading it: here it offers to downgrade Newman to version 4.

In the project: overrides took the count from 19 to 11, including the critical one. The rest are listed in the README.

4. Strategy and scenarios

Q49. When would you choose Postman over a code framework like REST Assured?

Postman when testers and developers want to explore, share and run collections with little setup. Code when tests need real logic, shared libraries and refactoring tools. Many teams use both.

Q50. What are Postman's weaknesses for automation?

JSON files that merge badly, little reuse between scripts, and no real debugger or refactoring support. Large collections get hard to maintain.

Q51. Why does this project have unit tests as well as a collection?

They reach what Postman can't easily send, such as a 65 KB body, and neither layer caught every deliberate bug.

In the project: the table in section 9 of the design document.

Q52. How do you prove your tests can fail?

Put bugs in on purpose and check that something goes red.

In the project: 14 deliberate bugs, all caught.

Q53. A collection passes locally and fails in CI. What do you check?

The environment and its variables, data left by earlier runs, the address, the token, and time zones. Then compare the raw requests.

Q54. A test passes alone and fails in a full run. Why?

An earlier request changed something it depends on, usually a shared variable or shared data.

Q55. How do you test an API that changes shared data?

Create what the test needs, use unique values, and clean up, or give each run its own copy.

In the project: a private server per run, so there's nothing to clean.

Q56. How would you test order creation by hand before automating?

Happy path first, then each rule, then access, then the read-back. The manual cases become the automated ones.

In the project: `docs/manual-test-cases.md`, with the same IDs as the tests.

Q57. What did you find while testing this API?

`GET //` crashed the server, because `new URL` threw outside any `try`. A quantity of `1e308` gave a total of `null`. Both came from asking what input the code never expected.

Q58. How do you test for a server crash?

Send the bad request, then a good one. The bad answer alone doesn't prove the server survived.

In the project: `TC-142` sends `GET //`, then checks `/health`.

Q59. What would you add next here?

Tests against a deployed environment, an HTML report, and a load test of the order endpoint.

Q60. What does this project not prove?

Anything about load or real network behaviour. It runs on localhost with one user at a time, against an API built for the project.